

QWASM

A Restricted, Verifiable Execution Layer for Operator Programs

Karel Capek

Abstract

QWASM is a restricted execution layer designed to serve as the uniform, verifiable target for operator-based computation in the QVM stack.

It provides a deterministic and auditable intermediate representation between high-level operator programs (QFM/QFP) and heterogeneous execution backends (CPU, QPU, and future accelerators). This document specifies the QWASM design goals, module format, instruction semantics, runtime ABI, validation rules, determinism guarantees, and integration pathways.

Contents

1	Introduction	6
1.1	Motivation	6
1.2	What QWASM Is	6
1.3	What QWASM Is Not	6
1.4	Relation to WebAssembly	7
1.5	Position in the QVM Stack	7
1.6	Determinism as a Primary Requirement	7
1.7	Verifiability and Validation	7
1.8	Security and Governance Alignment	8
1.9	Portability and Backend Independence	8
1.10	Scope of This Document	8
1.11	Structure of the Specification	8
2	Design Goals and Threat Model	9
2.1	Primary Design Goals	9
2.2	Secondary Design Goals	9
2.3	Explicit Non-Goals	9
2.4	Threat Model Overview	10
2.5	Malicious or Careless Program Authors	10
2.6	Compromised or Untrusted Execution Backends	10
2.7	Policy Circumvention Attempts	10
2.8	State Corruption and Leakage	11
2.9	Non-Determinism as a Threat	11
2.10	Denial-of-Service and Resource Exhaustion	11
2.11	Out-of-Scope Threats	11
2.12	Design Implications	11
2.13	Preparation for Module Format Specification	12

3	Module Format and Validation	12
3.1	Module as the Unit of Execution	12
3.2	High-Level Module Structure	12
3.3	Module Header	12
3.4	Capability Declaration Section	13
3.5	Resource and Bounds Declaration	13
3.6	Instruction Section	13
3.7	Interface and Metadata Section	13
3.8	Immutability and Integrity	14
3.9	Static Validation Overview	14
3.10	Control Flow Validation	14
3.11	Resource Consistency Validation	14
3.12	Capability Consistency Validation	15
3.13	Determinism Validation	15
3.14	Validation Outcomes	15
3.15	Preparation for Instruction Semantics	15
4	Instruction Set Semantics	15
4.1	Instruction Design Philosophy	15
4.2	Instruction Classes Overview	16
4.3	Control Flow Instructions	16
4.4	State Access Instructions	16
4.5	State Update Semantics	16
4.6	Operator Application Instructions	17
4.7	Aggregation and Reduction Instructions	17
4.8	Synchronization and Barrier Instructions	17
4.9	Runtime Interaction Instructions	17
4.10	Prohibited Instruction Categories	18
4.11	Instruction-Level Capability Enforcement	18
4.12	Deterministic Execution Semantics	18
4.13	Error and Exception Handling	18
4.14	Instruction Semantics and Validation Coupling	18
4.15	Preparation for Memory and State Model	19
5	Memory and State Model	19
5.1	Separation of Memory and State	19
5.2	Execution Memory	19
5.3	Typed State Elements	19
5.4	State Regions and Partitioning	20
5.5	State Access Rules	20
5.6	Deterministic Update Semantics	20
5.7	Ephemeral Versus Persistent State	20
5.8	Initialization and Finalization	20
5.9	No Direct Addressing	21
5.10	Memory Isolation	21
5.11	Interaction with Instruction Semantics	21
5.12	State Integrity and Validation	21

5.13	Preparation for Runtime Interfaces	21
6	Runtime ABI and System Calls	22
6.1	Principles of the Runtime ABI	22
6.2	ABI Versioning and Compatibility	22
6.3	System Call Categories	22
6.4	Execution Context Query Calls	22
6.5	State and Metadata Reporting Calls	23
6.6	Synchronization and Coordination Calls	23
6.7	Controlled Termination Calls	23
6.8	Prohibited System Calls	23
6.9	Capability Enforcement at the ABI Level	24
6.10	Deterministic Behavior of System Calls	24
6.11	Error Handling and ABI Violations	24
6.12	Audit and Trace Integration	24
6.13	Isolation from Host Environment	24
6.14	Preparation for Determinism Guarantees	25
7	Determinism and Reproducibility	25
7.1	Definition of Determinism	25
7.2	Sources of Non-Determinism	25
7.3	Deterministic Control Flow	26
7.4	Deterministic State Evolution	26
7.5	Backend-Independence of Semantics	26
7.6	Reproducibility Across Deployments	26
7.7	Reproducibility Under Distribution	26
7.8	Role of Validation in Determinism	27
7.9	Runtime Enforcement of Determinism	27
7.10	Trace-Based Verification	27
7.11	Reproducibility Versus Exploration	27
7.12	Failure Modes and Determinism	28
7.13	Formal Guarantees	28
7.14	Preparation for Sandboxing and Capabilities	28
8	Sandboxing and Capabilities	28
8.1	Purpose of Sandboxing	28
8.2	Execution Context Isolation	29
8.3	Absence of Ambient Authority	29
8.4	Capability Model Overview	29
8.5	Static Declaration of Capabilities	29
8.6	Capability Granularity	29
8.7	Capability Enforcement	30
8.8	Non-Transferability of Capabilities	30
8.9	No Capability Escalation	30
8.10	Sandboxed Interaction with Runtime Services	30
8.11	Isolation from Host Resources	31
8.12	Side-Channel Considerations	31

8.13	Sandbox Integrity and Failure Handling	31
8.14	Preparation for Backend Integration	31
9	Backends: CPU and QPU Integration	31
9.1	Backend Abstraction Boundary	32
9.2	Uniform Execution Semantics	32
9.3	CPU Backend Integration	32
9.4	QPU Backend Integration	32
9.5	Partitioning and Work Decomposition	32
9.6	Synchronization Between Backend Units	33
9.7	Backend Resource Management	33
9.8	Handling Backend Failure	33
9.9	Verification of Backend Correctness	33
9.10	Isolation of Backend Effects	34
9.11	Heterogeneous Backend Environments	34
9.12	Backend Selection and Policy	34
9.13	Future Backend Classes	34
9.14	Preparation for Toolchain and Verification	34
10	Toolchain, Compiler, and Verifier	35
10.1	Role of the Toolchain	35
10.2	Source Languages and Frontends	35
10.3	Compilation Pipeline	35
10.4	Explicit Bound Inference	35
10.5	Capability Inference and Declaration	36
10.6	Determinism Preservation in Compilation	36
10.7	Verifier Architecture	36
10.8	Separation of Compiler and Verifier	36
10.9	Verifier Determinism	37
10.10	Verification Artifacts	37
10.11	Reproducible Builds	37
10.12	Trust and Update Model	37
10.13	Failure Modes and Diagnostics	37
10.14	Preparation for Audit and Tracing	38
11	Audit, Tracing, and Result Sealing	38
11.1	Purpose of Audit and Tracing	38
11.2	Execution Traces	38
11.3	Trace Granularity	38
11.4	Deterministic Trace Generation	39
11.5	Trace Integrity	39
11.6	Audit Records	39
11.7	Result Sealing	39
11.8	Failure Sealing	39
11.9	Long-Term Verification	40
11.10	Independent Verification	40
11.11	Privacy Considerations	40

11.12	Retention and Archiving	40
11.13	Audit as Deterrence	40
11.14	Preparation for Reference Programs	41
12	Examples and Reference Programs	41
12.1	Role of Reference Programs	41
12.2	Minimal Valid QWASM Module	41
12.3	Example: Bounded State Update	41
12.4	Example: Operator Application	41
12.5	Example: Aggregation and Reduction	42
12.6	Example: Controlled Termination	42
12.7	Example: Deterministic Failure	42
12.8	Example: Capability Violation (Rejected Module)	42
12.9	Example: Non-Deterministic Construct (Rejected)	43
12.10	Reference Patterns	43
12.11	Anti-Patterns	43
12.12	Use in Testing and Certification	43
12.13	Extensibility of Examples	43
12.14	Preparation for Conclusion	44
13	Conclusion and Outlook	44
13.1	Summary of Contributions	44
13.2	Role Within the QVM Architecture	44
13.3	Security and Trust Implications	44
13.4	Extensibility Within Constraints	45
13.5	Deliberate Limits	45
13.6	Future Research Directions	45
13.7	Relationship to the Covenant	45
13.8	Closing Statement	45

1 Introduction

QWASM is the execution layer that bridges abstract operator programs and concrete computational hardware within the QVM stack. It serves as a restricted, verifiable, and deterministic intermediate representation designed specifically for operator-based computation governed by QFM and QFP.

This document specifies QWASM as a system in its own right: its purpose, design constraints, and role within the broader QVM architecture.

1.1 Motivation

Operator-based computation requires an execution substrate that differs fundamentally from general-purpose virtual machines.

Conventional execution environments prioritize flexibility, expressiveness, and performance. While suitable for many applications, these properties conflict with the requirements of QVM, which include:

- strict determinism,
- enforceable execution bounds,
- auditability of computation,
- compatibility with heterogeneous backends.

QWASM is introduced to meet these requirements without relying on ad-hoc restrictions or post-hoc verification.

1.2 What QWASM Is

QWASM is a restricted execution layer designed to:

- act as the uniform execution target for operator programs,
- enable static validation prior to execution,
- support deterministic execution across platforms,
- integrate directly with QVM Runtime enforcement mechanisms.

It is an intermediate representation rather than a general-purpose programming language.

1.3 What QWASM Is Not

QWASM is intentionally limited in scope.

It is not:

- a Turing-complete general-purpose virtual machine,
- a sandbox for arbitrary user code,
- a substitute for QFM, QFP, or QVM Runtime.

Expressiveness is constrained by design in order to preserve verifiability and control.

1.4 Relation to WebAssembly

QWASM draws conceptual inspiration from WebAssembly but diverges substantially in intent and semantics.

Key distinctions include:

- a reduced and domain-specific instruction set,
- absence of unrestricted memory access,
- explicit alignment with operator semantics,
- mandatory integration with policy and capability checks.

QWASM may be implemented atop existing WebAssembly runtimes, but its execution semantics are defined independently of any particular WASM implementation.

1.5 Position in the QVM Stack

Within the QVM architecture:

- QFM defines the mathematical structure of operators,
- QFP defines authorization, safety, and policy constraints,
- QWASM defines executable form and validation,
- the QVM Runtime orchestrates execution over time,
- QPU and CPU backends perform physical computation.

QWASM is the narrow waist of the execution pipeline: all computation passes through it, and all execution guarantees depend on it.

1.6 Determinism as a Primary Requirement

Determinism is a foundational requirement of QWASM.

Given:

- an identical QWASM module,
- identical execution context and policy,
- identical input state,

execution must produce identical results, independent of backend, parallelism, or timing.

Non-deterministic behavior is treated as a violation rather than a feature.

1.7 Verifiability and Validation

QWASM is designed to be statically verifiable prior to execution.

Validation ensures that:

- execution bounds are explicit,
- control flow is analyzable,
- memory and state access are constrained,
- required capabilities are declared.

Modules that cannot be fully validated are rejected.

1.8 Security and Governance Alignment

QWASM is not security-neutral.

Its design explicitly supports:

- enforcement of QFP policies,
- audit and trace generation,
- revocation and termination mechanisms.

Security properties are enforced structurally rather than assumed.

1.9 Portability and Backend Independence

QWASM decouples execution semantics from physical hardware.

The same QWASM module may be executed on:

- CPU-based interpreters or JITs,
- QPU-accelerated backends,
- future specialized execution units.

Portability does not compromise determinism or auditability.

1.10 Scope of This Document

This document specifies:

- QWASM design goals and threat model,
- module format and validation rules,
- instruction semantics,
- runtime ABI and system interfaces,
- determinism and audit guarantees.

It does not specify:

- high-level operator languages,
- orchestration policies,
- governance or ethical constraints.

These are defined in other documents of the QVM series.

1.11 Structure of the Specification

The remainder of this document proceeds as follows.

The next chapter defines the design goals and threat model underlying QWASM. Subsequent chapters specify the module format, instruction set, memory and state model, runtime interfaces, backend integration, tooling, and audit mechanisms, concluding with examples and future directions.

2 Design Goals and Threat Model

QWASM is designed under the assumption that execution environments are adversarial, complex, and prone to misuse if not explicitly constrained. This chapter defines the primary design goals of QWASM and the threat model against which it is constructed.

Design goals and threat assumptions are treated as first-class specification elements rather than implicit background conditions.

2.1 Primary Design Goals

The design of QWASM is guided by a small set of non-negotiable goals.

Determinism. Execution must be deterministic with respect to declared inputs, execution context, and policy. Timing, backend selection, parallelism, or physical deployment must not affect observable results.

Verifiability. All executable artifacts must be statically verifiable prior to execution. The absence of complete verification is grounds for rejection.

Bounded Execution. All computation must be explicitly bounded in time, space, and scope of effect. Unbounded loops, recursion, or resource growth are prohibited by construction.

Auditability. Execution must produce sufficient structured metadata to allow post-execution audit, verification, and attribution without re-running the computation.

Backend Independence. Execution semantics must be independent of the physical execution backend, including CPUs, QPUs, or future accelerators.

Policy Alignment. QWASM must integrate directly with QFP policy enforcement and capability systems rather than relying on external controls.

2.2 Secondary Design Goals

In addition to primary goals, QWASM pursues several secondary objectives.

These include:

- predictable performance characteristics,
- minimal semantic surface area,
- ease of static analysis and tooling,
- resistance to accidental misuse.

Secondary goals must never compromise primary goals.

2.3 Explicit Non-Goals

Certain properties commonly associated with execution environments are explicitly rejected.

QWASM does not aim to provide:

- Turing completeness,

- unrestricted dynamic code generation,
- implicit concurrency or scheduling,
- direct access to system resources.

These omissions are intentional and central to the design.

2.4 Threat Model Overview

The QWASM threat model assumes that threats may originate from multiple sources, including users, developers, operators, and infrastructure.

Threats are categorized according to the aspect of the system they target.

2.5 Malicious or Careless Program Authors

QWASM assumes that executable modules may be authored with malicious intent or insufficient care.

Potential threats include:

- attempts to bypass execution bounds,
- construction of pathological control flow,
- exploitation of undefined or underspecified behavior,
- concealment of intent through obfuscation.

Static validation is the primary defense against these threats.

2.6 Compromised or Untrusted Execution Backends

Execution backends are assumed to be fallible or untrusted.

Threats include:

- incorrect instruction implementation,
- non-deterministic behavior under load,
- side-channel leakage,
- partial execution or silent failure.

QWASM semantics are defined independently of backend behavior, and runtime enforcement detects deviation.

2.7 Policy Circumvention Attempts

QWASM assumes that attempts will be made to circumvent QFP policy constraints.

These attempts may involve:

- misreporting resource usage,
- fragmenting computation to evade limits,
- abusing permitted instructions for unintended effects.

QWASM integrates policy checks at the instruction and module level to prevent circumvention.

2.8 State Corruption and Leakage

Distributed execution introduces risks to state integrity and confidentiality.

Threats include:

- unauthorized state access,
- partial state corruption,
- cross-context state leakage.

QWASM restricts state access patterns and relies on the runtime for isolation and verification.

2.9 Non-Determinism as a Threat

In QWASM, non-determinism itself is treated as a threat.

Sources of non-determinism include:

- race-dependent behavior,
- timing-based branching,
- backend-specific execution order.

The instruction set and execution model are designed to eliminate these sources.

2.10 Denial-of-Service and Resource Exhaustion

QWASM assumes that modules may attempt to exhaust resources.

Threats include:

- excessive iteration within declared bounds,
- pathological memory access patterns,
- abuse of aggregation or synchronization primitives.

Explicit resource accounting and bounded constructs mitigate these threats.

2.11 Out-of-Scope Threats

Certain threats are explicitly considered out of scope for QWASM.

These include:

- physical attacks on hardware,
- compromise of cryptographic primitives,
- failures of external governance institutions.

While relevant to overall system security, these are addressed outside the QWASM specification.

2.12 Design Implications

The design goals and threat model jointly imply a constrained execution environment.

As a result:

- expressiveness is sacrificed for predictability,

- convenience is sacrificed for verifiability,
 - performance is bounded by enforceability.
- These trade-offs are deliberate and foundational.

2.13 Preparation for Module Format Specification

With design goals and threats defined, the next chapter specifies the concrete QWASM module format and validation rules through which these goals are enforced at load time.

3 Module Format and Validation

QWASM modules are immutable, self-contained execution artifacts designed to be statically validated prior to execution. The module format encodes not only executable instructions, but also explicit declarations of bounds, capabilities, and semantic intent.

This chapter specifies the structure of a QWASM module and the validation rules that determine whether a module is admissible for execution.

3.1 Module as the Unit of Execution

A QWASM module is the minimal unit of execution within the QVM stack.

A module:

- is immutable once created,
- is validated in its entirety before execution,
- executes within a single execution context,
- produces auditable and attributable outcomes.

Partial or incremental loading of executable content is not permitted.

3.2 High-Level Module Structure

A QWASM module consists of a fixed set of ordered sections.

At a minimum, a valid module includes:

- a module header,
- a capability declaration section,
- a resource and bounds declaration section,
- an instruction section,
- an interface and metadata section.

Sections appear in a canonical order to simplify validation.

3.3 Module Header

The module header uniquely identifies the module and its execution requirements.

The header includes:

- a module identifier,

- a version identifier,
- a reference to the QWASM specification version,
- a cryptographic hash of the module contents.

Modules referencing unsupported specification versions are rejected.

3.4 Capability Declaration Section

Capabilities define what the module is permitted to do.

This section explicitly declares:

- required operator classes,
- permitted state access patterns,
- allowed runtime services and syscalls.

Undeclared capabilities are implicitly denied. Capabilities may not be requested dynamically.

3.5 Resource and Bounds Declaration

All resource usage must be declared prior to execution.

Declarations include:

- maximum instruction count,
- memory and state access limits,
- aggregation and synchronization bounds,
- execution tick limits.

Bounds are treated as hard limits. Modules exceeding declared bounds during execution are deterministically terminated.

3.6 Instruction Section

The instruction section contains the executable body of the module.

Properties of the instruction section include:

- structured, analyzable control flow,
- absence of unbounded loops or recursion,
- explicit termination paths.

Self-modifying code and indirect jumps are prohibited.

3.7 Interface and Metadata Section

This section defines how the module interacts with the runtime and other components.

It includes:

- declared inputs and outputs,
- versioned metadata for audit and attribution,

- optional semantic annotations.

Metadata does not affect execution semantics but is included in audit records.

3.8 Immutability and Integrity

Once constructed, a QWASM module is immutable.

Any modification to:

- instructions,
- declarations,
- metadata,

results in a distinct module identity and requires re-validation.

Integrity is enforced through cryptographic identification and validation.

3.9 Static Validation Overview

Static validation is mandatory prior to execution.

Validation verifies:

- structural correctness of the module format,
- completeness of capability declarations,
- internal consistency of bounds,
- analyzability of control flow.

Modules that fail validation are rejected without partial execution.

3.10 Control Flow Validation

Control flow must be statically analyzable.

Validation ensures:

- all loops have statically bounded iteration counts,
- all branches lead to a termination point,
- no unreachable or ambiguous execution paths exist.

Unstructured or data-dependent control flow is prohibited.

3.11 Resource Consistency Validation

Declared resource bounds must be internally consistent.

Validation checks include:

- instruction counts matching control flow bounds,
- memory access patterns respecting declared limits,
- aggregation operations remaining within bounds.

Inconsistent declarations result in rejection.

3.12 Capability Consistency Validation

All instructions must be authorized by declared capabilities.

Validation enforces:

- one-to-one correspondence between instruction classes and capabilities,
- absence of implicit or inferred privileges,
- explicit denial of undeclared actions.

Capabilities cannot be escalated at runtime.

3.13 Determinism Validation

Validation ensures that execution semantics are deterministic.

This includes:

- prohibition of timing-dependent constructs,
- fixed ordering of operations,
- absence of backend-dependent behavior.

Any potential source of non-determinism results in rejection.

3.14 Validation Outcomes

Validation produces one of the following outcomes:

- acceptance for execution,
- rejection with diagnostic information.

There is no conditional or partial acceptance.

3.15 Preparation for Instruction Semantics

With the module format and validation rules defined, the next chapter specifies the semantics of the QWASM instruction set upon which these validation guarantees depend.

4 Instruction Set Semantics

The QWASM instruction set defines the executable semantics of operator-based computation within the QVM stack. Instructions are designed to be minimal, explicit, and analyzable, prioritizing determinism and verifiability over expressive power.

This chapter specifies the classes of QWASM instructions and their semantics, as well as the constraints governing their use.

4.1 Instruction Design Philosophy

QWASM instructions are not general-purpose operations.

They are designed to:

- directly correspond to operator-level primitives,
- admit static analysis and bounded execution,

- avoid implicit side effects,
- remain independent of backend implementation details.

Each instruction has a precisely defined effect on state and execution context.

4.2 Instruction Classes Overview

The QWASM instruction set is divided into distinct classes.

Primary instruction classes include:

- control flow instructions,
- state access and update instructions,
- operator application instructions,
- aggregation and reduction instructions,
- synchronization and barrier instructions,
- runtime interaction instructions.

Each class is governed by specific validation and capability requirements.

4.3 Control Flow Instructions

Control flow in QWASM is explicitly structured.

Permitted control flow instructions include:

- bounded loops with statically known iteration counts,
- conditional branches based on local state,
- explicit termination instructions.

Unbounded loops, recursion, indirect jumps, and data-dependent control flow that cannot be statically bounded are prohibited.

4.4 State Access Instructions

State access instructions mediate interaction with the distributed state managed by the QVM Runtime.

These instructions:

- read from declared state regions,
- write only to explicitly authorized regions,
- operate on well-defined state elements.

Direct memory addressing or pointer arithmetic is not permitted.

4.5 State Update Semantics

State updates are deterministic and ordered.

Update semantics include:

- atomic updates within a logical tick,

- fixed ordering of updates,
 - prohibition of race-dependent behavior.
- Partial updates are not externally observable.

4.6 Operator Application Instructions

Operator application instructions apply QFM-defined operators to state.

These instructions:

- invoke operator kernels with resolved parameters,
- apply effects to assigned state partitions,
- produce deterministic partial results.

Operators may not be dynamically constructed or modified at runtime.

4.7 Aggregation and Reduction Instructions

Aggregation instructions combine partial results into global quantities.

Supported operations include:

- summation,
- norm and inner product evaluation,
- trace-like reductions.

Aggregation order is fixed and independent of physical execution order.

4.8 Synchronization and Barrier Instructions

Synchronization instructions coordinate distributed execution.

These include:

- explicit barrier instructions,
- synchronization points for state consistency,
- coordination of aggregation phases.

Implicit synchronization is prohibited.

4.9 Runtime Interaction Instructions

Runtime interaction instructions allow limited interaction with the QVM Runtime.

Permitted interactions include:

- querying execution bounds,
- reporting execution metadata,
- requesting controlled termination.

Instructions cannot modify runtime policy or execution context parameters.

4.10 Prohibited Instruction Categories

Certain instruction categories are explicitly prohibited.

These include:

- self-modifying instructions,
- dynamic code loading,
- system calls outside the declared ABI,
- direct hardware interaction.

Prohibited instructions are rejected at validation time.

4.11 Instruction-Level Capability Enforcement

Each instruction requires specific capabilities.

Validation enforces:

- correspondence between instruction usage and declared capabilities,
- denial of implicit privilege escalation,
- absence of capability inference.

Instructions without explicit authorization cannot be executed.

4.12 Deterministic Execution Semantics

Instruction semantics are deterministic by definition.

Determinism is ensured by:

- fixed evaluation order,
- absence of timing-dependent behavior,
- elimination of backend-specific effects.

Any deviation is treated as a violation.

4.13 Error and Exception Handling

QWASM does not support general-purpose exception handling.

Errors are handled through:

- deterministic termination,
- explicit error reporting,
- sealing of execution state.

Error handling does not introduce alternative execution paths.

4.14 Instruction Semantics and Validation Coupling

Instruction semantics are tightly coupled with validation rules.

Validation ensures that:

- all instruction effects are bounded,

- instruction sequences are analyzable,
- semantic constraints are enforced prior to execution.

Execution assumes successful validation.

4.15 Preparation for Memory and State Model

With instruction semantics defined, the next chapter specifies the memory and state model upon which these instructions operate.

5 Memory and State Model

The QWASM memory and state model defines how executable instructions interact with computational state. Unlike general-purpose execution environments, QWASM does not expose raw memory. Instead, it operates on structured state elements defined and managed by the QVM Runtime.

This chapter specifies the abstractions, access rules, and lifecycle of state within QWASM execution.

5.1 Separation of Memory and State

QWASM distinguishes between transient execution memory and persistent computational state.

- **Execution memory** is used for local, temporary values within a module.
- **Runtime state** represents the distributed computational state managed by the QVM Runtime.

Only the runtime state has semantic significance beyond a single execution tick.

5.2 Execution Memory

Execution memory is strictly local to a QWASM module.

Properties include:

- bounded size declared at validation time,
- no persistence across executions,
- no sharing between modules.

Execution memory is automatically cleared upon termination.

5.3 Typed State Elements

Runtime state is composed of typed state elements.

Each element:

- has a declared type and structure,
- belongs to a specific execution context,
- is associated with access permissions.

Untyped or dynamically typed state access is not permitted.

5.4 State Regions and Partitioning

State elements are organized into regions.

Regions:

- group related state elements,
- define access and update boundaries,
- may correspond to distributed partitions.

QWASM instructions operate on declared regions only.

5.5 State Access Rules

State access is explicit and constrained.

Rules include:

- read and write access must be declared,
- access is limited to assigned regions,
- no aliasing between writable regions.

Violation of access rules results in deterministic termination.

5.6 Deterministic Update Semantics

State updates follow deterministic semantics.

Updates are:

- applied atomically at synchronization points,
- ordered independently of physical execution,
- free of race conditions.

Intermediate states are not externally observable.

5.7 Ephemeral Versus Persistent State

By default, runtime state accessed through QWASM is ephemeral.

Persistent state:

- must be explicitly declared,
- requires authorization via policy and capabilities,
- is subject to sealing and audit.

QWASM itself does not manage persistence; it relies on the runtime.

5.8 Initialization and Finalization

State initialization occurs prior to execution.

Initialization includes:

- allocation of declared regions,
- application of initial conditions,

- verification of access permissions.

Finalization occurs upon termination and includes sealing of results.

5.9 No Direct Addressing

QWASM does not permit direct addressing or pointer-based access.

State is referenced through:

- symbolic identifiers,
- statically resolved offsets,
- runtime-managed handles.

This prevents memory corruption and unauthorized access.

5.10 Memory Isolation

Memory and state are isolated across execution contexts.

Isolation guarantees that:

- one module cannot access another's state,
- stale handles are invalid,
- terminated contexts leave no residual access.

Isolation is enforced by the runtime, not by convention.

5.11 Interaction with Instruction Semantics

Instruction semantics are defined relative to the state model.

Instructions assume:

- validated access permissions,
- consistent state layout,
- deterministic update ordering.

Instruction behavior is undefined outside these assumptions and therefore disallowed.

5.12 State Integrity and Validation

State integrity is preserved through validation and runtime checks.

Integrity guarantees include:

- absence of out-of-bounds access,
- consistency between declared and actual state usage,
- detection of corruption or mismatch.

Integrity violations result in termination and audit.

5.13 Preparation for Runtime Interfaces

With the memory and state model defined, the next chapter specifies the runtime ABI and system interfaces through which QWASM interacts with the QVM Runtime.

6 Runtime ABI and System Calls

QWASM interacts with the QVM Runtime exclusively through a restricted and well-defined Application Binary Interface (ABI). This ABI exposes only the minimal set of system calls required for controlled execution, monitoring, and termination.

This chapter specifies the QWASM runtime ABI, the semantics of permitted system calls, and the constraints governing their use.

6.1 Principles of the Runtime ABI

The runtime ABI is designed according to the following principles:

- **Minimality:** only strictly necessary functionality is exposed.
- **Determinism:** system calls must not introduce non-deterministic behavior.
- **Capability-Governed Access:** all calls require explicit authorization.
- **Observability:** all calls are auditable runtime events.

The ABI is not extensible by user modules.

6.2 ABI Versioning and Compatibility

Each QWASM module declares the ABI version it targets.

The runtime enforces:

- strict matching of supported ABI versions,
- rejection of unknown or deprecated interfaces,
- forward compatibility only where explicitly specified.

ABI versioning is treated as part of the security boundary.

6.3 System Call Categories

System calls are grouped into a small number of categories.

Permitted categories include:

- execution context queries,
- state and metadata reporting,
- synchronization and coordination,
- controlled termination.

Any system call outside these categories is prohibited.

6.4 Execution Context Query Calls

These calls allow a module to query bounded properties of its execution context.

Examples include:

- remaining instruction or tick budget,
- declared execution bounds,

- execution context identifiers.

Returned values are deterministic and read-only.

6.5 State and Metadata Reporting Calls

Modules may report metadata to the runtime for audit and monitoring.

Reporting calls include:

- emission of execution summaries,
- reporting of intermediate metrics,
- declaration of completion status.

Reported data does not affect execution semantics.

6.6 Synchronization and Coordination Calls

Synchronization calls allow a module to participate in runtime-managed coordination.

Permitted operations include:

- entering declared barrier points,
- signaling completion of local phases,
- participating in aggregation coordination.

Synchronization does not grant access to external state or execution control.

6.7 Controlled Termination Calls

Modules may request controlled termination.

Termination requests:

- must specify a termination reason,
- result in deterministic shutdown,
- trigger sealing and audit procedures.

Termination requests cannot be withdrawn once issued.

6.8 Prohibited System Calls

The runtime ABI explicitly prohibits system calls that would expose or modify runtime internals.

Prohibited calls include:

- modification of execution context parameters,
- invocation of external system resources,
- dynamic capability escalation,
- spawning of additional execution contexts.

Prohibited calls are rejected at validation time.

6.9 Capability Enforcement at the ABI Level

Each system call requires specific capabilities.

Validation and runtime enforcement ensure:

- calls are authorized by declared capabilities,
- unauthorized calls cannot be invoked,
- no implicit capability inheritance occurs.

Capabilities cannot be inferred or negotiated at runtime.

6.10 Deterministic Behavior of System Calls

System calls must be deterministic in their observable effects.

This includes:

- fixed return values for identical context,
- absence of timing-based variation,
- independence from backend scheduling.

Non-deterministic calls are incompatible with QWASM.

6.11 Error Handling and ABI Violations

Errors related to ABI usage are handled deterministically.

Possible outcomes include:

- immediate termination,
- emission of structured error records,
- sealing of partial execution state.

There is no undefined behavior in the ABI.

6.12 Audit and Trace Integration

All system calls are included in execution traces.

Audit records include:

- call identifiers and parameters,
- execution tick at which the call occurred,
- outcome and side effects.

Trace data enables reconstruction of execution behavior.

6.13 Isolation from Host Environment

The runtime ABI provides no direct access to the host environment.

QWASM modules cannot:

- perform I/O,
- access clocks or randomness,

- interact with the network or filesystem.

All such interaction must occur outside QWASM.

6.14 Preparation for Determinism Guarantees

With the runtime ABI defined, the next chapter formalizes determinism and reproducibility guarantees provided by QWASM across executions and deployments.

7 Determinism and Reproducibility

Determinism is a foundational property of QWASM. It ensures that execution results are a function solely of declared inputs, module contents, and execution context, rather than of timing, hardware, or environmental variation.

This chapter defines determinism and reproducibility in the context of QWASM, specifies the guarantees provided by the execution model, and describes how these properties are enforced and verified.

7.1 Definition of Determinism

A QWASM execution is deterministic if, given:

- an identical QWASM module,
- identical declared inputs and initial state,
- identical execution context and policy,

the execution produces identical observable outcomes.

Observable outcomes include:

- final state values,
- sealed results,
- execution metadata and traces.

Differences in execution time or resource scheduling are not considered observable outcomes.

7.2 Sources of Non-Determinism

QWASM explicitly identifies and eliminates common sources of non-determinism.

Prohibited sources include:

- timing-based branching or measurement,
- data races or implicit concurrency,
- backend-dependent instruction ordering,
- access to external entropy or randomness.

Any construct that could introduce such variation is disallowed.

7.3 Deterministic Control Flow

Control flow in QWASM is deterministic by construction.

This is ensured through:

- statically bounded loops,
- explicit and analyzable branching,
- prohibition of indirect or data-dependent jumps.

Control flow decisions depend only on local, validated state.

7.4 Deterministic State Evolution

State evolution under QWASM execution is deterministic.

Determinism is preserved through:

- fixed ordering of state updates,
- atomic application of updates at synchronization points,
- elimination of race conditions.

Physical execution order does not affect logical state transitions.

7.5 Backend-Independence of Semantics

QWASM semantics are defined independently of execution backend.

This implies that:

- CPU, QPU, or accelerator implementations must conform to the same semantics,
- parallel execution must not alter results,
- backend-specific optimizations must preserve observable behavior.

Backend deviation constitutes a correctness violation.

7.6 Reproducibility Across Deployments

Reproducibility extends beyond single execution instances.

A QWASM module is reproducible if its execution can be repeated:

- on different hardware,
- in different deployment environments,
- at different points in time,

while producing identical observable outcomes.

Reproducibility requires preservation of module identity, execution context, and policy versioning.

7.7 Reproducibility Under Distribution

Distributed execution does not compromise reproducibility.

This is achieved by:

- deterministic partitioning of state,

- explicit synchronization points,
- fixed aggregation semantics.

Network latency or message ordering does not affect logical execution.

7.8 Role of Validation in Determinism

Static validation is the first line of defense for determinism.

Validation ensures:

- absence of non-deterministic constructs,
- bounded control flow and resource usage,
- consistency of state access declarations.

Execution assumes that validation has succeeded.

7.9 Runtime Enforcement of Determinism

The runtime enforces determinism during execution.

Enforcement mechanisms include:

- monitoring of instruction and resource usage,
- detection of deviation from declared bounds,
- deterministic termination upon violation.

There is no attempt to recover from non-deterministic behavior.

7.10 Trace-Based Verification

Execution traces enable post hoc verification of determinism.

Trace records include:

- instruction-level summaries,
- state transition checkpoints,
- system call invocations.

Traces allow independent verification without re-execution.

7.11 Reproducibility Versus Exploration

QWASM distinguishes determinism from exploratory variability.

Exploratory execution:

- must be explicitly authorized,
- records all sources of variability,
- produces reproducible sets rather than single outcomes.

Exploration does not weaken deterministic guarantees of standard execution.

7.12 Failure Modes and Determinism

Failures are treated as deterministic outcomes.

Given identical conditions, failures:

- occur at the same execution point,
- produce identical error records,
- result in identical sealed states.

Failure does not introduce ambiguity.

7.13 Formal Guarantees

The determinism guarantees of QWASM may be summarized as follows:

- identical inputs imply identical outcomes,
- execution is independent of timing and hardware,
- violations are detectable and attributable.

These guarantees form the basis for trust in QWASM-based computation.

7.14 Preparation for Sandboxing and Capabilities

With determinism and reproducibility defined, the next chapter specifies the sandboxing and capability mechanisms that enforce isolation and prevent privilege escalation during execution.

8 Sandboxing and Capabilities

QWASM enforces strict isolation between executable modules, runtime state, and the host environment. This isolation is achieved through a combination of sandboxing and a capability-based authorization model.

This chapter specifies how QWASM modules are sandboxed, how capabilities are declared and enforced, and how privilege escalation is prevented.

8.1 Purpose of Sandboxing

Sandboxing ensures that a QWASM module can affect only those parts of the system that it is explicitly authorized to affect.

Sandboxing serves to:

- protect the runtime and host environment,
- isolate execution contexts from one another,
- limit the impact of faulty or malicious modules.

Isolation is treated as a correctness requirement, not merely a security feature.

8.2 Execution Context Isolation

Each QWASM module executes within a dedicated execution context.

Isolation guarantees that:

- modules cannot access each other's state,
- execution memory is not shared across contexts,
- termination of one context does not affect others.

Context boundaries are enforced by the runtime and are not visible to the module.

8.3 Absence of Ambient Authority

QWASM provides no ambient authority.

Modules do not inherit:

- host system privileges,
- runtime control authority,
- implicit access to state or services.

All authority must be explicitly granted through capabilities.

8.4 Capability Model Overview

Capabilities are explicit, declarative grants of permission.

A capability specifies:

- the class of operations permitted,
- the scope of state or resources accessible,
- applicable bounds and constraints.

Capabilities are declared in the module and validated prior to execution.

8.5 Static Declaration of Capabilities

All required capabilities must be declared statically.

Static declaration ensures:

- full visibility of module intent,
- complete validation before execution,
- prohibition of dynamic privilege acquisition.

Modules requesting undeclared capabilities are rejected.

8.6 Capability Granularity

Capabilities are defined with fine granularity.

Granularity includes distinctions between:

- read versus write access,
- local versus global state interaction,

- operator application versus aggregation.

Coarse-grained, all-purpose capabilities are intentionally avoided.

8.7 Capability Enforcement

Capabilities are enforced at multiple levels.

Enforcement includes:

- validation-time checks,
- instruction-level authorization,
- runtime monitoring of usage.

Unauthorized actions result in deterministic termination.

8.8 Non-Transferability of Capabilities

Capabilities are non-transferable.

A module may not:

- delegate capabilities to another module,
- export capabilities through state or metadata,
- acquire capabilities indirectly.

Capabilities exist only within the execution context to which they are granted.

8.9 No Capability Escalation

Capability escalation is prohibited.

Modules cannot:

- request additional capabilities at runtime,
- infer authority from execution behavior,
- combine permitted actions to achieve prohibited effects.

Any attempt at escalation constitutes a violation.

8.10 Sandboxed Interaction with Runtime Services

Interaction with runtime services is sandboxed through the ABI.

The sandbox ensures that:

- only permitted system calls are available,
- calls cannot escape declared bounds,
- responses cannot leak sensitive information.

Runtime services do not expose internal state.

8.11 Isolation from Host Resources

QWASM modules have no direct access to host resources.

Prohibited access includes:

- file systems,
- network interfaces,
- clocks, timers, or randomness sources.

Any interaction with external resources must occur outside QWASM.

8.12 Side-Channel Considerations

While absolute elimination of side channels is infeasible, QWASM minimizes their impact.

Mitigations include:

- elimination of timing-dependent semantics,
- fixed execution ordering,
- absence of data-dependent resource allocation.

Side-channel resistance is treated as a design goal rather than an absolute guarantee.

8.13 Sandbox Integrity and Failure Handling

Sandbox integrity is continuously monitored.

Violations result in:

- immediate termination of the offending module,
- sealing of partial execution state,
- emission of audit records.

There is no recovery path that preserves partial privileges.

8.14 Preparation for Backend Integration

With sandboxing and capabilities defined, the next chapter describes how QWASM integrates with execution backends, including CPU and QPU targets, while preserving isolation and enforcement guarantees.

9 Backends: CPU and QPU Integration

QWASM defines execution semantics independently of any physical computation device. Execution backends are responsible for realizing these semantics on actual hardware while preserving determinism, auditability, and enforcement of declared constraints.

This chapter specifies how QWASM integrates with CPU and QPU backends and the requirements imposed on all backend implementations.

9.1 Backend Abstraction Boundary

The boundary between QWASM and execution backends is strict.

QWASM specifies:

- instruction semantics,
- state and memory behavior,
- ordering and synchronization rules.

Backends implement these semantics without modification or extension. Behavior not explicitly defined by QWASM is disallowed.

9.2 Uniform Execution Semantics

All backends must implement identical observable semantics.

Uniformity requires that:

- instruction effects are equivalent across backends,
- state transitions follow the same logical ordering,
- termination conditions are consistent.

Backend-specific optimizations may not alter observable outcomes.

9.3 CPU Backend Integration

CPU backends provide reference execution for QWASM.

Characteristics include:

- sequential or explicitly managed parallel execution,
- direct mapping of QWASM instructions to host instructions,
- deterministic scheduling enforced by the runtime.

CPU backends serve as the baseline for correctness verification.

9.4 QPU Backend Integration

QPU backends accelerate operator application and aggregation.

Integration requirements include:

- mapping QWASM operator instructions to QPU kernels,
- deterministic decomposition of work units,
- controlled aggregation of partial results.

QPU backends must not introduce probabilistic or non-deterministic behavior into observable results.

9.5 Partitioning and Work Decomposition

Execution may be partitioned across backend units.

Partitioning rules require:

- deterministic partition boundaries,

- reproducible mapping from state to work units,
- explicit synchronization points.

Dynamic or adaptive partitioning based on runtime conditions is prohibited.

9.6 Synchronization Between Backend Units

Backend units must respect QWASM synchronization semantics.

This includes:

- honoring explicit barrier instructions,
- completing local computation before aggregation,
- preventing observation of intermediate states.

Synchronization behavior is defined by QWASM, not by backend heuristics.

9.7 Backend Resource Management

Backends must operate within declared resource bounds.

Resource management includes:

- enforcing instruction and tick limits,
- respecting memory and state access bounds,
- reporting resource usage accurately.

Backends may not exceed declared bounds, even if resources are available.

9.8 Handling Backend Failure

Backend failures are treated as execution events.

Failure handling requires:

- deterministic detection of failure,
- termination or rollback according to runtime policy,
- emission of structured failure records.

Partial or silent failure is not permitted.

9.9 Verification of Backend Correctness

Backend implementations must be verifiable.

Verification mechanisms include:

- cross-backend result comparison,
- execution trace validation,
- reference execution on CPU backends.

Deviation from expected semantics constitutes a backend defect.

9.10 Isolation of Backend Effects

Backends operate within the same sandbox constraints as QWASM modules.

This ensures:

- no direct access to unauthorized state,
- no leakage of intermediate data,
- no modification of execution context.

Backend privilege is strictly limited to execution duties.

9.11 Heterogeneous Backend Environments

QWASM supports heterogeneous execution environments.

A single execution may involve:

- multiple CPU backends,
- one or more QPU backends,
- fallback execution paths.

Heterogeneity must not compromise determinism or auditability.

9.12 Backend Selection and Policy

Backend selection is governed by runtime policy.

Policies may consider:

- declared capabilities,
- resource availability,
- trust level of backends.

Modules do not influence backend selection.

9.13 Future Backend Classes

The QWASM backend interface is designed to accommodate future backend classes.

New backends must:

- implement existing QWASM semantics,
- pass verification and audit requirements,
- integrate without expanding the ABI surface.

Backward compatibility does not justify semantic deviation.

9.14 Preparation for Toolchain and Verification

With backend integration defined, the next chapter specifies the toolchain, compiler, and verifier infrastructure that produces and validates QWASM modules.

10 Toolchain, Compiler, and Verifier

QWASM is not authored directly by humans. It is the output of a controlled toolchain that transforms higher-level operator programs into validated, executable modules. This chapter specifies the structure of the QWASM toolchain, the role of compilers, and the function of verification systems.

The toolchain is treated as part of the trusted computing base and is subject to explicit constraints.

10.1 Role of the Toolchain

The QWASM toolchain mediates between intent and execution.

Its responsibilities include:

- translating high-level operator descriptions into QWASM modules,
- enforcing syntactic and semantic constraints,
- producing artifacts suitable for validation and audit.

Manual construction of QWASM modules is discouraged and may be restricted by policy.

10.2 Source Languages and Frontends

QWASM supports multiple source languages and frontends.

These may include:

- operator languages derived from QFM,
- domain-specific languages for structured computation,
- verified intermediate representations.

All frontends must compile to the same QWASM semantics.

10.3 Compilation Pipeline

Compilation to QWASM proceeds through a fixed pipeline.

Typical stages include:

- parsing and semantic analysis,
- operator normalization,
- resource and bounds inference,
- instruction selection and scheduling,
- module assembly and annotation.

Stages may not be reordered in a way that obscures intent or validation.

10.4 Explicit Bound Inference

The compiler is responsible for inferring and declaring execution bounds.

This includes:

- instruction count bounds,

- memory and state usage limits,
- aggregation and synchronization bounds.

Bounds must be conservative. Underestimation is a compilation error.

10.5 Capability Inference and Declaration

The compiler infers required capabilities from program semantics.

Inferred capabilities:

- must be explicitly declared in the module,
- may not exceed the minimum required,
- are subject to validation and policy approval.

Capabilities may not be inferred implicitly at runtime.

10.6 Determinism Preservation in Compilation

The compiler must preserve determinism.

This requires:

- avoidance of non-deterministic transformations,
- deterministic instruction ordering,
- explicit handling of parallelizable constructs.

Compiler optimizations must not alter observable semantics.

10.7 Verifier Architecture

Verification is a mandatory, standalone phase.

The verifier:

- analyzes complete QWASM modules,
- checks structural and semantic validity,
- enforces capability and bound correctness.

Verification must succeed before any execution is authorized.

10.8 Separation of Compiler and Verifier

The compiler and verifier are logically distinct.

This separation ensures:

- independent detection of errors,
- resistance to compiler bugs or compromise,
- clearer attribution of responsibility.

A module accepted by the compiler but rejected by the verifier is invalid.

10.9 Verifier Determinism

Verifier behavior must itself be deterministic.

Given an identical module, the verifier must:

- produce identical validation outcomes,
- emit identical diagnostic information.

Verifier non-determinism undermines trust.

10.10 Verification Artifacts

Verification produces explicit artifacts.

These include:

- validation certificates,
- diagnostic reports,
- hashes of verified module contents.

Artifacts are included in audit records.

10.11 Reproducible Builds

The toolchain supports reproducible builds.

Reproducibility requires:

- deterministic compilation outputs,
- versioned compiler and verifier binaries,
- recorded compilation parameters.

Reproducible builds enable independent verification.

10.12 Trust and Update Model

Toolchain components evolve over time.

Updates must:

- be versioned and documented,
- preserve backward compatibility where possible,
- undergo independent review.

Toolchain trust is earned through transparency.

10.13 Failure Modes and Diagnostics

Toolchain failures are explicit and diagnosable.

Failure handling includes:

- clear error reporting,
- preservation of intermediate artifacts,
- prohibition of silent fallback behavior.

Ambiguous failure is treated as a defect.

10.14 Preparation for Audit and Tracing

With the toolchain defined, the next chapter specifies audit, tracing, and result sealing mechanisms that preserve accountability across execution and time.

11 Audit, Tracing, and Result Sealing

QWASM execution is designed to be inspectable, attributable, and verifiable after the fact. Audit, tracing, and result sealing mechanisms ensure that execution outcomes can be trusted over time, even in adversarial or distributed environments.

This chapter specifies how execution is recorded, how traces are produced, and how results are sealed for long-term verification.

11.1 Purpose of Audit and Tracing

Audit and tracing provide accountability.

They enable:

- verification that execution followed declared semantics,
- attribution of actions to authorized modules and policies,
- detection of misuse or deviation.

Auditability is treated as a first-class execution requirement.

11.2 Execution Traces

Each QWASM execution produces an execution trace.

A trace includes:

- module identity and hash,
- execution context and policy identifiers,
- summaries of instruction execution,
- state transition checkpoints,
- system call invocations.

Traces are structured, deterministic, and reproducible.

11.3 Trace Granularity

Trace granularity is defined by policy.

Granularity may vary between:

- coarse summaries for routine execution,
- fine-grained traces for sensitive or critical computation.

Granularity changes do not affect execution semantics.

11.4 Deterministic Trace Generation

Trace generation is deterministic.

Given identical execution conditions:

- identical traces are produced,
- trace ordering and contents are consistent,
- no timing-dependent variation is present.

Trace determinism is essential for independent verification.

11.5 Trace Integrity

Trace integrity is protected cryptographically.

Integrity guarantees include:

- cryptographic hashing of trace segments,
- chaining of trace records,
- detection of modification or omission.

Tampered traces are invalid.

11.6 Audit Records

Audit records summarize execution for governance and review.

Records include:

- execution purpose and authorization,
- declared capabilities and bounds,
- outcomes and termination status.

Audit records are derived from traces but are optimized for review.

11.7 Result Sealing

Upon successful completion, execution results are sealed.

Sealing includes:

- cryptographic binding of results to module identity,
- inclusion of execution context and policy,
- linkage to corresponding trace records.

Sealed results cannot be altered without detection.

11.8 Failure Sealing

Failures are sealed in the same manner as successes.

Failure sealing ensures:

- failures are attributable and reproducible,
- partial results are not misrepresented,

- error conditions are preserved for analysis.

Failure does not escape accountability.

11.9 Long-Term Verification

Audit and sealing support long-term verification.

Verification may occur:

- immediately after execution,
- during governance review,
- years after execution.

Long-term trust does not depend on continued runtime availability.

11.10 Independent Verification

Third parties may independently verify execution.

Verification may involve:

- recomputation using preserved modules and inputs,
- validation of traces and hashes,
- cross-checking against reference backends.

Independent verification strengthens trust.

11.11 Privacy Considerations

Audit and tracing must respect privacy and confidentiality.

This requires:

- minimization of sensitive data in traces,
- controlled access to audit records,
- separation of operational metadata from content.

Transparency does not imply indiscriminate disclosure.

11.12 Retention and Archiving

Audit artifacts must be retained according to policy.

Retention policies consider:

- legal and governance requirements,
- storage and integrity constraints,
- future verification needs.

Deletion or alteration of required records is prohibited.

11.13 Audit as Deterrence

The existence of audit and trace mechanisms acts as a deterrent.

Knowing that execution is permanently attributable discourages misuse and careless deployment.

11.14 Preparation for Reference Programs

With audit and sealing mechanisms defined, the next chapter presents reference programs and examples illustrating correct QWASM usage.

12 Examples and Reference Programs

This chapter presents illustrative examples and reference program patterns for QWASM. The purpose of these examples is not to exhaustively cover all possible use cases, but to demonstrate correct usage, common patterns, and typical failure modes.

Examples are intentionally minimal and schematic. They emphasize structure, declarations, and constraints rather than performance or optimization.

12.1 Role of Reference Programs

Reference programs serve multiple purposes:

- to clarify the abstract semantics defined in previous chapters,
- to provide canonical patterns for tooling and verification,
- to act as regression tests for implementations.

They are normative examples of correct QWASM usage.

12.2 Minimal Valid QWASM Module

A minimal QWASM module consists of:

- a valid module header,
- explicit capability declarations,
- explicit resource and bounds declarations,
- a terminating instruction sequence.

Even the simplest module must be fully declarative and verifiable.

12.3 Example: Bounded State Update

This example illustrates a bounded update to a declared state element.

Key properties:

- read and write access are explicitly declared,
- the number of update operations is statically bounded,
- execution terminates deterministically.

Such patterns are typical for controlled state transformation.

12.4 Example: Operator Application

This example demonstrates application of a predefined operator.

The example highlights:

- static resolution of operator identity,

- explicit parameter binding,
- deterministic effect on assigned state regions.

Dynamic operator construction is intentionally absent.

12.5 Example: Aggregation and Reduction

Aggregation examples illustrate safe combination of partial results.

Properties include:

- fixed aggregation order,
- declared aggregation bounds,
- explicit synchronization points.

Aggregation does not depend on execution parallelism.

12.6 Example: Controlled Termination

This example shows explicit, controlled termination.

Termination includes:

- declaration of termination reason,
- sealing of partial or final results,
- emission of audit metadata.

Abrupt or implicit termination is prohibited.

12.7 Example: Deterministic Failure

Failures are also executable outcomes.

This example demonstrates:

- detection of a bound violation,
- deterministic termination at a fixed point,
- sealed failure records.

Failure behavior is reproducible and auditable.

12.8 Example: Capability Violation (Rejected Module)

Some examples illustrate invalid modules.

A capability violation example includes:

- use of an instruction without declared authorization,
- rejection at validation time,
- diagnostic output explaining the violation.

Such examples define negative space in the specification.

12.9 Example: Non-Deterministic Construct (Rejected)

This example demonstrates rejection of non-deterministic constructs.

Rejected patterns include:

- data-dependent control flow without static bounds,
- timing-based branching,
- backend-specific behavior.

The example illustrates how validation enforces determinism.

12.10 Reference Patterns

Several reference patterns recur in QWASM programs:

- map-reduce style computation with fixed bounds,
- staged execution with explicit barriers,
- read-compute-write cycles with atomic updates.

These patterns are preferred over ad-hoc constructions.

12.11 Anti-Patterns

The following anti-patterns are explicitly discouraged:

- attempting to simulate unbounded loops,
- encoding control flow in data structures,
- relying on backend-specific optimization behavior.

Anti-patterns may pass superficial validation but violate design intent.

12.12 Use in Testing and Certification

Reference programs are used for:

- backend conformance testing,
- verifier correctness validation,
- regression testing across versions.

They provide a shared baseline for implementation comparison.

12.13 Extensibility of Examples

The set of examples is expected to grow.

New examples must:

- adhere to the same validation rules,
- remain minimal and illustrative,
- avoid introducing new semantics implicitly.

Examples do not redefine the specification.

12.14 Preparation for Conclusion

With examples and reference programs defined, the final chapter summarizes the role of QWASM within the QVM architecture and outlines future directions.

13 Conclusion and Outlook

QWASM defines a restricted, verifiable execution layer that enables operator-based computation to be executed with determinism, accountability, and policy alignment across heterogeneous hardware environments.

This document has specified QWASM as a complete system: from design goals and threat model, through module structure and instruction semantics, to runtime interfaces, backend integration, tooling, audit, and examples.

13.1 Summary of Contributions

The core contributions of QWASM may be summarized as follows:

- a deterministic and backend-independent execution model,
- a statically verifiable module format with explicit bounds,
- a capability-based authorization system without ambient authority,
- a narrow and auditable runtime ABI,
- trace-based audit and result sealing mechanisms.

Together, these elements form a coherent execution substrate for QVM.

13.2 Role Within the QVM Architecture

Within the QVM stack, QWASM serves as the execution “waist”.

It connects:

- high-level operator definitions and policies,
- low-level execution backends and accelerators,
- governance and audit mechanisms.

By constraining this interface, QWASM enables innovation above and below it without compromising system-wide guarantees.

13.3 Security and Trust Implications

QWASM shifts trust away from opaque execution environments toward explicit, verifiable artifacts.

Trust is established through:

- static validation rather than runtime heuristics,
- deterministic semantics rather than probabilistic assurances,
- auditability rather than obscurity.

This approach aligns execution with long-term governance and stewardship requirements.

13.4 Extensibility Within Constraints

QWASM is intentionally conservative, but not static.

Future extensions may include:

- additional operator classes,
- new backend integrations,
- richer audit and verification tooling.

All extensions must preserve determinism, verifiability, and bounded execution.

13.5 Deliberate Limits

Equally important are the limits QWASM does not seek to cross.

It does not aim to:

- become a general-purpose execution platform,
- absorb orchestration or governance responsibilities,
- trade safety for convenience or performance.

These limits are foundational, not temporary.

13.6 Future Research Directions

Several areas warrant further investigation:

- formal semantics and machine-checked proofs,
- certified backend implementations,
- integration with zero-knowledge verification,
- long-term archival of execution artifacts.

Research in these areas can strengthen, but must not dilute, the core model.

13.7 Relationship to the Covenant

While this document is technical in nature, its design is consistent with the QVM Covenant.

QWASM enforces, at the execution layer, the principles of bounded power, accountability, and refusal articulated at the normative level.

13.8 Closing Statement

QWASM is not defined by what it allows, but by what it refuses to permit.

In doing so, it provides a stable foundation for powerful computation that can be trusted not because it is unrestricted, but because it is constrained.

Its success will be measured not by breadth of adoption alone, but by the clarity with which it preserves intent, responsibility, and verifiable outcome.